Unidad 2:

Acceso a la información

FICHA TÉCNICA DE LA UNIDAD

Unidad trabajo	UT3. Administración y control de acceso: Privilegios, roles y seguridad
Resultados de Aprendizaje	**RA3.** Implanta métodos de control de acceso utilizando asistentes, herramientas gráficas y comandos del lenguaje del sistema gestor.
Criterios de Evaluación	**CE3.a** Se han creado vistas personalizadas para cada tipo de usuario. **CE3.b** Se han creado sinónimos de tablas y vistas. **CE3.c** Se han definido y eliminado cuentas de usuario. **CE3.d** Se han identificado los privilegios sobre las bases de datos y sus elementos. **CE3.e** Se han agrupado y desagrupado privilegios **CE3.f** Se han asignado y eliminado privilegios a usuarios. **CE3.g** Se han asignado y eliminado grupos de privilegios a usuarios. **CE3.h** Se ha garantizando el cumplimiento de los requisitos de seguridad.
Tiempo	8 horas lectivas

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. ADMINISTRACIÓN DE CUENTAS DE USUARIO

2.1. Sentencia de Creación de Cuentas (CREATE USER)

2.2. Sintaxis y Plugins de Autenticación en MySQL 8.0

2.3. Persistencia en el Diccionario de Datos (mysql.user)

2.4. Eliminación de Cuentas (DROP USER)

3. EL SISTEMA DE PRIVILEGIOS DE MYSQL

3.1. Niveles de Granularidad de Permisos

3.1.1. Privilegios a Nivel de Base de Datos

3.1.2. Privilegios a Nivel de Tabla (mysql.tables_priv)

3.1.3. Privilegios a Nivel de Columna (mysql.columns_priv)

3.1.4. Privilegios sobre Rutinas y Procedimientos (mysql.procs_priv)

3.2. Delegación de Capacidades (WITH GRANT OPTION)

3.3. Revocación de Permisos (REVOKE)

3.4. Auditoría de Concesiones (SHOW GRANTS)

3.5. Sincronización de Tablas de Concesión (FLUSH PRIVILEGES)

4. GESTIÓN AVANZADA MEDIANTE ROLES

4.1. Concepto de Rol en MySQL 8.0

4.2. Roles Administrativos de Factoría en Entornos de Gestión

4.3. Ciclo Operativo de Trabajo con Roles (RBAC Lifecycle)

4.4. Estrategias de Activación: Automatización vs Manual

5. VISTAS EN BASES DE DATOS

5.1. ¿Qué es una Vista y Ventajas Principales?

5.2. Crear o Modificar una Vista (CREATE VIEW)

5.3. Eliminar una Vista (DROP VIEW)

5.4. Renombrar una Vista (RENAME TABLE)

5.5. Consultar Vistas y su Definición (SHOW)

5.6. Vistas con Contenido Actualizable (INSERT, UPDATE, DELETE)

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

En el ámbito de la administración de sistemas informáticos, los Servidores de Gestión de Bases de Datos (SGBD) custodian el activo más crítico de cualquier organización: sus datos. Por consiguiente, delimitar con precisión milimétrica las fronteras de acceso de cada entidad que interactúa con el motor de base de datos no es una tarea meramente accesoria, sino una responsabilidad mandatoria de todo Administrador de Sistemas (SysAdmin).

Toda política de seguridad corporativa implantada sobre un SGBD debe regirse de manera estricta bajo el **Principio del Menor Privilegio (Principle of Least Privilege)**. Este axioma de la seguridad informática dictamina que a un usuario, aplicación o proceso se le deben conceder única y exclusivamente los privilegios mínimos necesarios para completar sus tareas encomendadas, durante el periodo de tiempo justo, y en el ámbito espacial idóneo. Minimizar los accesos reduce drásticamente la superficie de ataque del servidor, mitigando el impacto de inyecciones de código (SQLi), elevaciones de privilegios ilegítimas o errores humanos accidentales.

Esta unidad de trabajo no es una isla de conocimiento aislada; por el contrario, actúa como un núcleo interconector de diversas áreas críticas del ciclo de Grado Superior en ASIR:

- **Implantación de Sistemas Operativos (ISO):** Se asimila de forma directa a la gestión multiusuario de sistemas Linux (análisis de archivos de seguridad /etc/passwd y /etc/shadow) y al control de accesos mediante ACLs (Access Control Lists).
- **Planificación y Administración de Redes (PAR):** Enlaza con la seguridad de sockets TCP/IP, el aislamiento de puertos de escucha del motor (como el puerto por defecto 3306) sobre interfaces o subredes específicas (interfaces loopback, VLANs de gestión), y la implantación de túneles de comunicación cifrados mediante protocolos criptográficos TLS/SSL.
- **Servicios de Red e Internet (SRI) e Implantación de Aplicaciones Web (IAW):** Proporciona el marco de seguridad indispensable para la interconexión con entornos de desarrollo y producción (como scripts de automatización e infraestructura en Python). Al conectar una aplicación web o un script de monitorización Python (usando la librería mysql-connector-python o PyMySQL), el SysAdmin nunca debe emplear la cuenta root. Se instruirá en la creación de credenciales exclusivas no-root con los privilegios mínimos

(DML limitado y selectivo), garantizando que el compromiso lógico del servidor de aplicaciones web o del script de monitorización no otorgue el control del motor de bases de datos.

2. ADMINISTRACIÓN DE CUENTAS DE USUARIO

2.1. Sentencia de Creación de Cuentas (CREATE USER)

En MySQL, una cuenta de usuario se define mediante una estructura compuesta por dos partes diferenciadas e indisolubles: 'nombre_usuario'@'host'. La omisión o alteración de cualquiera de estas dos partes rompe el esquema de validación de accesos del motor de base de datos.

```
-- Sintaxis general base para la provisión de cuentas de usuario
CREATE USER 'nombre_usuario'@'host' IDENTIFIED WITH
mysql_native_password BY 'contraseña';
```

- **nombre_usuario:** Identificador alfanumérico que representa la cuenta dentro de la instancia.
- **host:** El parámetro de red que determina la ubicación física o lógica desde la cual el usuario tiene autorización explícita para abrir una sesión de red.
    - '%': Carácter comodín que indica que el usuario puede establecer conexión desde cualquier host o dirección IP externa (acceso remoto global).
    - 'localhost': Restringe la comunicación de forma absoluta, permitiendo únicamente conexiones procedentes de la misma máquina mediante sockets locales del sistema operativo, anulando el acceso por red TCP/IP externa.
    - '172.16.11.10': Permite acotar el acceso exclusivamente a direccionamiento de una máquina o pasarela de red específica de producción o gestión.

2.2. Sintaxis y Plugins de Autenticación en MySQL 8.0

MySQL 8.0 expande los mecanismos de control de acceso permitiendo diversificar los plugins que procesan la validación de las credenciales, ofreciendo algoritmos criptográficos más avanzados (como caching_sha2_password, el valor por defecto de la versión 8.0).

A continuación se detalla el comportamiento del servidor ante las diferentes variantes sintácticas aplicables al comando:

Cláusula de Autenticación	Descripción Técnica y Comportamiento del Motor
IDENTIFIED BY 'auth_string'	Crea la cuenta empleando el plugin criptográfico configurado por defecto en la directiva global del servidor (típicamente caching_sha2_password).
IDENTIFIED BY RANDOM PASSWORD	Genera una clave criptográficamente segura y aleatoria de forma automática, mostrándola en la consola del administrador para su inmediata comunicación.
IDENTIFIED WITH auth_plugin	Invoca un plugin específico de autenticación (por ejemplo, validación por sockets del sistema) sin requerir una contraseña almacenada internamente.
IDENTIFIED WITH auth_plugin BY 'auth_string'	Fuerza explícitamente el uso de un plugin específico junto con la contraseña en texto plano, la cual será procesada y hasheada por dicho plugin.
IDENTIFIED WITH auth_plugin BY RANDOM PASSWORD	Configura un plugin concreto y asigna una contraseña aleatoria de alta seguridad autogenerada en el momento de la ejecución.
IDENTIFIED WITH auth_plugin AS 'auth_string'	Permite pasar directamente la cadena ya cifrada o el hash precalculado de la contraseña, técnica habitual en tareas de migración y scripts SysAdmin.

```
-- EJEMPLO PRÁCTICO: Creación de cuentas con diferentes directivas de autenticación
-- 1. Cuenta para una aplicación interna que requiere compatibilidad con el formato clásico
CREATE USER 'app_legacy'@'172.16.11.10'
IDENTIFIED WITH mysql_native_password BY 'ClaveSegura2026!';
-- 2. Cuenta de administración temporal con contraseña autogenerada por el motor
CREATE USER 'admin_temp'@'%'
IDENTIFIED BY RANDOM PASSWORD;
```

Nota de administración histórica: En versiones antiguas de MySQL se utilizaba la sentencia SET PASSWORD = PASSWORD('new_secret') para cambiar claves; sin embargo, en MySQL

8.0 esta sintaxis está obsoleta y se prioriza la sentencia ALTER USER 'user'@'host' IDENTIFIED BY 'new_password';.

2.3. Persistencia en el Diccionario de Datos (mysql.user)

Cada vez que se ejecuta un comando CREATE USER, el SGBD valida la sintaxis y escribe de forma permanente un nuevo registro en el diccionario de datos del motor, específicamente en la tabla del sistema **mysql.user**. Esta tabla actúa como el repositorio de identidades globales, guardando los hashes de las claves, los plugins asignados y los privilegios globales (superusuario).

2.4. Eliminación de Cuentas (DROP USER)

Para revocar el acceso permanente de un usuario en el servidor, se debe purgar su registro del diccionario mediante la sentencia:

```
-- Eliminación explícita de una cuenta de usuario
DROP USER 'nombre_usuario'@'host';
```

3. EL SISTEMA DE PRIVILEGIOS DE MYSQL

3.1. Niveles de Granularidad de Permisos

Para cumplir con el *Principio del Menor Privilegio*, MySQL implementa una arquitectura jerárquica con cuatro niveles de granularidad bien definidos. A medida que descendemos de nivel, el control se vuelve más específico y restrictivo:

```
[Nivel Global] --> Afecta a todas las bases de datos (*.*)
|
[Nivel de Esquema] --> Afecta a una Base de Datos específica (mydb.*)
|
[Nivel de Tabla] --> Afecta a una Tabla concreta (mydb.mytbl)
|
[Nivel de Columna] --> Restringe la acción a campos específicos
```

3.1.1. Privilegios a Nivel de Base de Datos

Estos privilegios impactan sobre la totalidad de los objetos residentes dentro de un esquema (base de datos) determinado utilizando la nomenclatura base_de_datos.*.

```
-- Estructura base para la concesión a nivel de esquema
GRANT tipo_de_permiso ON base_de_datos.* TO 'nombre_usuario'@'host';
```

Los privilegios que se pueden acotar a nivel de esquema son:

- **SELECT, INSERT, UPDATE:** Operaciones de lectura y manipulación de datos sobre cualquier tabla de la base de datos.
- **CREATE:** Concede la facultad de estructurar y crear nuevas tablas en el esquema.
- **DROP:** Permite eliminar de forma permanente tablas de la base de datos.
- **ALTER:** Habilita la alteración estructural de tablas existentes. *Nota técnica:* Para ejecutar exitosamente ciertas operaciones de alteración, el motor exige que la cuenta posea también el privilegio CREATE.
- **EVENT:** Faculta al usuario para programar, modificar o eliminar tareas temporales automatizadas ejecutadas por el motor interno del SGBD (Event Scheduler).
- **LOCK TABLES:** Permite congelar o bloquear explícitamente el acceso concurrente a una tabla para realizar lecturas o volcados consistentes. Requisito: La cuenta requiere poseer explícitamente el permiso SELECT sobre dicha tabla para poder bloquearla.
- **REFERENCES:** Otorga la capacidad indispensable para trazar restricciones de clave foránea e integridad referencial entre tablas del esquema.

3.1.2. Privilegios a Nivel de Tabla (mysql.tables_priv)

Cuando el acceso debe restringirse a un objeto de datos específico, los permisos se declaran apuntando de forma unívoca a la tabla en cuestión. Internamente, el motor añade registros en la tabla del sistema **mysql.tables_priv**, aplicando las validaciones de acceso de forma directa sobre esa entidad.

```
-- Concesión de control absoluto sobre una tabla concreta
GRANT ALL ON mydb.mytbl TO 'someuser'@'somehost';
```

```
-- Concesión de accesos de manipulación acotados
GRANT SELECT, INSERT ON mydb.mytbl TO 'someuser'@'somehost';
```

Los privilegios aplicables a este nivel funcional se desglosan en:

- **Gestión Estructural:** CREATE (crear una tabla específica, permitiendo incluso conceder el permiso antes de que la tabla exista en disco), ALTER (modificarla) y DROP (eliminarla).
- **Manipulación de Datos DML:** SELECT, INSERT, UPDATE y DELETE. Al declararse a nivel de tabla, estas directivas afectan de forma automática a la totalidad de las columnas de dicha entidad.
- **Gestión de Vistas (Visiones Externas):** CREATE VIEW (crear vistas sobre la tabla) y SHOW VIEW (inspeccionar los metadatos y la sentencia de creación de la vista). *Nota:* El borrado o modificación de una vista requiere obligatoriamente que la cuenta ostente el permiso de DROP.
- **Otros Privilegios de Tabla:** INDEX (crear o eliminar índices), REFERENCES (claves foráneas) y TRIGGER (crear, alterar o purgar disparadores de auditoría asociados a la tabla).

3.1.3. Privilegios a Nivel de Columna (mysql.columns_priv)

Representa el máximo nivel de aislamiento de datos en MySQL. Permite restringir las operaciones básicas de datos a un subconjunto cerrado de columnas de una tabla. Estos permisos generan apuntes en la tabla del sistema **mysql.columns_priv** y, de forma concurrente, actualizan los descriptores en mysql.tables_priv.

```
-- Concesión DML restringida por columnas específicas de datos
GRANT SELECT (col1), INSERT (col1, col2) ON mydb.mytbl TO
'someuser'@'somehost';
```

Los únicos cuatro privilegios aceptados por la sintaxis a este nivel atómico son de forma estricta: SELECT, INSERT, UPDATE y REFERENCES.

3.1.4. Privilegios sobre Rutinas y Procedimientos (mysql.procs_priv)

Regulan la creación y ejecución de código procedural alojado en la base de datos (bloques de lógica de negocio). Dependiendo del alcance del comando, los cambios alteran la tabla mysql.procs_priv (para un elemento determinado), mysql.db (esquema completo) o mysql.user (global).

```
-- Otorgar derechos globales de creación sobre el esquema de datos
GRANT CREATE ROUTINE ON mydb.* TO 'someuser'@'somehost';
-- Otorgar derechos de ejecución individuales para una rutina crítica
GRANT EXECUTE ON PROCEDURE mydb.myproc TO 'someuser'@'somehost';
```

Clasificación de privilegios de rutinas:

- **Aplicables a cualquier rutina del ámbito:** CREATE ROUTINE (crear), ALTER ROUTINE (modificar o borrar de forma masiva) y EXECUTE (ejecución global).
- **Aplicables a una rutina concreta determinada:** EXECUTE (ejecutar únicamente esa función o procedimiento) y ALTER ROUTINE (modificar o borrar exclusivamente esa pieza de código de la rutina).

3.2. Delegación de Capacidades (WITH GRANT OPTION)

La inclusión de la cláusula WITH GRANT OPTION al término de una sentencia de privilegios faculta al usuario receptor para transferir a terceros los mismos permisos exactos que él ha recibido, delegando de esta manera la cadena de control.

```
-- Concesión de privilegios administrativos de rutinas con capacidad de delegación
GRANT CREATE ROUTINE, ALTER ROUTINE ON mydb.* TO
'someuser'@'somehost' WITH GRANT OPTION;
```

3.3. Revocación de Permisos (REVOKE)

Para retirar accesos operativos de forma parcial o total a una cuenta sin necesidad de destruir su identidad en el servidor, se utiliza el comando REVOKE. La declaración del espacio de nombres y los objetos debe guardar una correspondencia exacta con el GRANT inicial:

```
-- Sintaxis general base para revocar accesos
REVOKE tipo_de_permiso ON base_de_datos.tabla FROM
```

```
'nombre_usuario'@'host';
-- EJEMPLO PRÁCTICO SYSADMIN: Revocar privilegios DML globales
REVOKE INSERT ON *.* FROM 'usuario'@'localhost';
```

Reglas de Control para la Revocación:

1. Para quitar un permiso a un usuario, la cuenta que ejecuta el comando debe poseer dicho privilegio y haberlo obtenido originalmente con la opción de delegación (WITH GRANT OPTION).
2. Si un administrador posee el permiso de sistema **CREATE USER**, está facultado para limpiar de forma masiva todos los permisos de una cuenta mediante la sentencia:

```
-- Purga absoluta de privilegios y facultades de delegación de una cuenta
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'user'@'localhost';
```

3.4. Auditoría de Concesiones (SHOW GRANTS)

Todo SysAdmin debe auditar con regularidad el estado consolidado de los accesos del servidor. Para inspeccionar los permisos efectivos de una cuenta o de un rol determinado se ejecuta:

```
-- Inspección de permisos de una cuenta de usuario
SHOW GRANTS FOR 'nombre_usuario'@'host';
```

3.5. Sincronización de Tablas de Concesión (FLUSH PRIVILEGES)

Cuando los privilegios se asignan mediante los comandos estándar de control (GRANT, REVOKE, CREATE USER), el motor de MySQL actualiza el diccionario y recarga de forma inmediata las estructuras en memoria.

Sin embargo, si realizamos modificaciones directas utilizando sentencias DML manuales (como un UPDATE o INSERT sobre la base de datos nativa mysql de forma directa), los cambios no surtirán efecto de inmediato. En tales escenarios, es de obligado cumplimiento ejecutar:

```
-- Forzar la recarga en memoria de las tablas de privilegios
FLUSH PRIVILEGES;
```

Este comando sincroniza la memoria interna del servidor con los registros físicos del diccionario de datos de forma instantánea, aplicando los nuevos controles de seguridad sin interrumpir las conexiones ni requerir un reinicio del servicio de MySQL.

4. GESTIÓN AVANZADA MEDIANTE ROLES

4.1. Concepto de Rol en MySQL 8.0

A partir de la versión 8.0, MySQL implementa el control de acceso basado en roles (**RBAC - Role-Based Access Control**). Un rol se define técnicamente como una abstracción de identidad o contenedor intermedio que agrupa una colección de privilegios de cualquier nivel.

En entornos corporativos complejos, este modelo simplifica la administración de la seguridad: en lugar de mantener y auditar individualmente los privilegios de decenas de técnicos, los permisos se asignan a roles estandarizados (p. ej., "Desarrollador", "Cajero", "Auditor") y posteriormente los usuarios simplemente heredan estos contenedores de permisos.

4.2. Roles Administrativos de Factoría en Entornos de Gestión

Las herramientas avanzadas de administración de sistemas de bases de datos (como *MySQL Workbench*) exponen de forma nativa una serie de roles de gestión preconfigurados listos para su uso operativo en la industria:

Rol en Workbench	Funciones y Competencias de Administración Asignadas
DBA	Administrador de Base de Datos General. Concede la totalidad de los privilegios del servidor de forma ilimitada.
MaintenanceAdmin	Otorga capacidades orientadas al mantenimiento físico del servidor, optimización y desfragmentación de tablas.
ProcessAdmin	Permite supervisar los hilos de ejecución de la instancia y finalizar procesos concurrentes (KILL) de otros usuarios.

UserAdmin	Faculta la provisión y borrado de cuentas de usuarios, así como el restablecimiento de contraseñas.
SecurityAdmin	Permite administrar inicios de sesión a nivel de servidor, ejecutando sentencias de concesión y revocación globales.
MonitorAdmin	Perfil de mínimos privilegios indispensables para herramientas de monitorización y recolección de métricas de rendimiento.
DBManager	Concede control total para la creación, modificación y eliminación de bases de datos o esquemas completos.
DBDesigner	Permite diseñar esquemas y realizar ingeniería inversa sobre cualquier modelo estructural de datos.
ReplicationAdmin	Otorga privilegios para la puesta en marcha, monitorización y administración de topologías en replicación (Master/Slave).
BackupAdmin	Proporciona los derechos y variables del sistema mínimos necesarios para realizar respaldos eficientes de los datos.
Custom	Estado descriptor utilizado para identificar colecciones de privilegios a medida asignados de forma directa.

4.3. Ciclo Operativo de Trabajo con Roles (RBAC Lifecycle)

Para desplegar un sistema seguro de control basado en roles, se debe completar de manera ordenada un flujo operativo de cuatro fases secuenciales:

```
[1. CREATE ROLE]        ---> [2. GRANT TO ROLE] --->
[3. GRANT ROLE TO USER] ---> [4. ACTIVATE ROLE]
```

Paso 1: Creación del Rol en el Servidor

Se utiliza el comando CREATE ROLE, pudiendo opcionalmente verificar su existencia previa.

```
-- Provisión de nuevos contenedores de privilegios (Roles)
CREATE ROLE IF NOT EXISTS mantenimiento_basico, mantenimiento_total,
consulta_basica;
```

Regla de nomenclatura y comportamiento interno: Es posible definir un rol siguiendo el patrón 'nombre_rol'@'host'. Sin embargo, a diferencia de las cuentas de usuario, el uso del carácter

% se interpreta como una cadena **literal** identitaria y no como un patrón comodín de red. Si se omite la sección del host, el SGBD asignará '%' por defecto.

Internamente, al registrar un rol, MySQL inserta una fila en la tabla de identidades global del diccionario. Herramientas gráficas como MySQL Workbench lo listarán en su interfaz junto a los usuarios corrientes, pero con una restricción de protección: **su cuenta de sistema nace bloqueada y su contraseña expirada de forma permanente**, anulando cualquier posibilidad de inicio de sesión directo con la identidad de dicho rol.

```
-- Creación de un rol con host de conexión de red explícito
CREATE ROLE IF NOT EXISTS 'rol_dev'@'%';
```

Paso 2: Dotación de Privilegios al Rol

Se asocian los permisos necesarios al rol utilizando la sintaxis estándar de GRANT, pero situando el rol como el elemento destino del comando.

```
-- Ejemplos de asignación a diferentes niveles de granularidad
GRANT EXECUTE ON *.* TO mantenimiento_total; -- Ámbito Global de Sistema
GRANT SELECT ON employees.* TO mantenimiento_basico; -- Ámbito de Base de Datos
GRANT SELECT ON employees.departments TO consulta_basica; -- Ámbito de Tabla
```

Paso 3: Vinculación del Rol a las Cuentas de Usuario

Una vez configurado el rol con sus respectivos permisos, se procede a conceder dicho rol a los técnicos u operarios de datos que lo requieran.

```
-- Vincular los roles a cuentas de usuario preexistentes en el sistema
GRANT mantenimiento_basico, consulta_basica TO 'user1'@'%';
GRANT mantenimiento_total TO 'user2'@'%';
```

Concepto Avanzado (Mandatory Roles): El administrador del sistema puede definir directivas de roles obligatorios (mandatory_roles), forzando al motor a asignar de forma automática determinados perfiles a toda conexión entrante, garantizando una línea base de seguridad homogénea.

Paso 4: Activación del Rol

Por diseño de seguridad nativo en MySQL 8.0, el hecho de que un usuario tenga un rol asignado no implica que disponga de sus privilegios inmediatamente al abrir una conexión. Si el usuario inicia sesión y ejecuta la función de diagnóstico de contexto:

```
-- Comprobación de roles activos en el hilo de conexión actual
SELECT CURRENT_ROLE();
```

El motor del sistema devolverá inicialmente el valor NONE. Por lo tanto, para poder hacer uso de las capacidades heredadas del rol, es imperativo proceder con su **activación**.

4.4. Estrategias de Activación: Automatización vs Manual

La activación de los roles asignados se puede gestionar implementando dos aproximaciones distintas según las necesidades operativas de la infraestructura:

A. Estrategia de Activación Automática (Por Defecto)

Configura la cuenta de usuario para que, en cada inicio de sesión entrante, el servidor active de forma automática la totalidad o parte de sus roles, haciendo los privilegios transparentes para las aplicaciones clientes.

Esta operación requiere ser ejecutada por un administrador del sistema que posea el privilegio CREATE USER (o derechos de modificación directa sobre la tabla del sistema mysql.default_roles):

```
-- 1. Forzar la activación automática de TODOS los roles asignados al iniciar sesión
SET DEFAULT ROLE ALL TO 'user1'@'%', 'user2'@'%';
-- 2. Configurar la activación automática por defecto de un único rol determinado
SET DEFAULT ROLE mantenimiento_total TO 'user2'@'%';
```

B. Estrategia de Activación Manual (Por Sesión en Tiempo de Ejecución)

Otorga al usuario el control operativo para activar o desactivar sus capacidades según las necesidades específicas de la tarea que vaya a realizar, cumpliendo de forma estricta con el principio de compartimentación de privilegios. Para interactuar con sus perfiles en tiempo de ejecución, el propio usuario conectado utiliza la instrucción SET ROLE:

```
-- Cambiar el contexto de la sesión actual de forma manual (Ejecutado por el usuario)
SET ROLE 'mantenimiento_total'; -- Activa el rol indicado y suspende el resto
SET ROLE ALL; -- Activa de forma simultánea todos los roles asignados
SET ROLE NONE; -- Desactiva todos los roles, devolviendo la sesión a mínimos privilegios
```

Existen variantes avanzadas aplicables a entornos complejos que permiten activar de forma masiva todos los perfiles de la cuenta a excepción de algunos explícitamente excluidos (SET ROLE ALL EXCEPT rol_name), dotando al SysAdmin de una gran versatilidad para orquestar entornos de bases de datos altamente seguros y compartimentados.

5. VISTAS EN BASES DE DATOS

5.1. ¿Qué es una Vista y Ventajas Principales?

Una vista es una tabla virtual creada a partir del resultado de una consulta SQL que se almacena en el diccionario de datos del gestor. A diferencia de una tabla física, la vista no duplica los datos en disco, sino que los obtiene dinámicamente de las tablas subyacentes en el momento en que es consultada.

Desde la perspectiva de la administración de sistemas y seguridad (CE3.a), las principales ventajas que aporta su utilización son:

- **Seguridad y control de acceso (Confidencialidad):** Permiten ocultar columnas sensibles (por ejemplo, salarios, contraseñas, datos personales) o filtrar filas específicas para usuarios o aplicaciones que no deban acceder a la tabla base completa.
- **Simplificación operativa:** Encapsulan consultas complejas que involucran múltiples combinaciones (JOINs), funciones de agregación o condiciones elaboradas en un objeto virtual reutilizable y fácil de consultar.
- **Independencia lógica de datos:** Si la estructura física de las tablas subyacentes sufre modificaciones, es posible ajustar la definición de la vista para mantener intacta la interfaz consumida por aplicaciones y usuarios finales.

5.2. Crear o Modificar una Vista (CREATE VIEW)

En MySQL, la instrucción `CREATE VIEW` permite registrar la especificación de una vista virtual. Para actualizar una vista preexistente sin necesidad de eliminarla previamente se emplea la cláusula complementaria `OR REPLACE`.

```
-- Sintaxis formal general en MySQL
CREATE
    [OR REPLACE]
    [ALGORITHM = {UNDEFINED | MERGE | TEMPTABLE}]
    [DEFINER = user]
    [SQL SECURITY { DEFINER | INVOKER }]
    VIEW nombre_vista [(lista_columnas)]
    AS sentencia_select
    [WITH [CASCADED | LOCAL] CHECK OPTION];
```

Parámetro / Cláusula	Descripción y Comportamiento
OR REPLACE	Si la vista ya existe en el esquema, se reemplaza su definición sin arrojar error ni requerir un DROP previo.
ALGORITHM	Define el mecanismo de procesamiento del motor: `MERGE` (combina el texto de la consulta externa con la vista), `TEMPTABLE` (materializa el resultado en una tabla temporal en memoria/disco) o `UNDEFINED` (MySQL selecciona el algoritmo más óptimo automáticamente).
SQL SECURITY	Determina el contexto de seguridad en la ejecución: `DEFINER` (ejecuta con los privilegios del creador) o `INVOKER` (ejecuta con los permisos del usuario que invoca la consulta).
WITH CHECK OPTION	Impide que sentencias de modificación DML (INSERT o UPDATE) introduzcan filas que no satisfagan la cláusula WHERE de la vista (variantes: `CASCADED` o `LOCAL`).

```
-- Ejemplo de definición con renombrado explícito de columnas:
CREATE OR REPLACE VIEW vista_pedidos_totales (id_pedido, fecha, cliente, importe_total) AS
SELECT
    p.codigo_pedido,
    p.fecha_pedido,
    c.nombre_cliente,
    SUM(dp.cantidad * dp.precio_unidad)
FROM cliente c
INNER JOIN pedido p ON c.codigo_cliente = p.codigo_cliente
INNER JOIN detalle_pedido dp ON p.codigo_pedido = dp.codigo_pedido
GROUP BY p.codigo_pedido;
```

5.3. Eliminar una Vista (DROP VIEW)

Para suprimir una o varias vistas del catálogo del sistema sin alterar los datos persistidos en las tablas base, se utiliza la sentencia `DROP VIEW`. Se recomienda incluir la cláusula preventiva `IF EXISTS`:

```
-- Eliminación segura de una o más vistas
DROP VIEW IF EXISTS nombre_vista [, otra_vista] [RESTRICT | CASCADE];
```

5.4. Renombrar una Vista (RENAME TABLE)

Dado que en el diccionario de datos de MySQL las vistas comparten espacio de nombres con las tablas, la modificación de su identificador se realiza mediante el comando `RENAME TABLE`:

```
-- Renombrar una vista en el catálogo
RENAME TABLE nombre_antiguo_vista TO nuevo_nombre_vista;
```

5.5. Consultar Vistas y su Definición (SHOW)

Para auditar qué vistas están desplegadas y examinar la sentencia SQL con la que fueron creadas, MySQL proporciona sentencias de introspección directas:

```
-- 1. Listar todas las tablas y vistas indicando su tipo (BASE TABLE o VIEW)
SHOW FULL TABLES;

-- 2. Filtrar para inspeccionar únicamente los objetos de tipo VIEW
SHOW FULL TABLES WHERE Table_type = 'VIEW';

-- 3. Visualizar la sentencia completa de definición DDL de una vista
SHOW CREATE VIEW nombre_vista;
```

5.6. Vistas con Contenido Actualizable (INSERT, UPDATE, DELETE)

En determinados casos y bajo condiciones estrictas, el SGBD permite ejecutar sentencias de manipulación de datos (`INSERT` , `UPDATE` , `DELETE`) a través de una vista para modificar directamente los datos de la tabla base subyacente.

Para que una vista sea **actualizable** (*updatable view*), debe existir una relación uno a uno entre las filas de la vista y las de la tabla original. La vista **no** puede contener:

- Funciones de agregación o resumen (`SUM` , `AVG` , `MIN` , `MAX` , `COUNT` , etc.).
- Cláusulas `GROUP BY` o `HAVING` .
- La cláusula `DISTINCT` .
- Operadores de conjuntos como `UNION` o `UNION ALL` .
- Subconsultas en la cláusula `SELECT` .

Condición obligatoria para inserciones: Para poder realizar un `INSERT` a través de una vista, todas las columnas de la tabla base que no admitan nulos y no tengan un valor por defecto (o clave primaria sin auto-incremento) deben formar parte obligatoriamente de la definición de la vista.